Karachi AQI Forecasting System

An End-to-End Machine Learning Pipeline for Air Quality Prediction

Final Project Report

Prepared by Hiba Asghar

Live application: aqi-predictor-jhpqumq2pnd6fud4vuzzfj.streamlit.app

Repository: github.com/hibaasghar30/aqi-predictor

Table of Contents

1. Executive Summary

This project delivers a complete, production-style system that forecasts the Air Quality Index (AQI) for Karachi, Pakistan, up to 72 hours in advance. The system covers the full machine learning lifecycle end to end: automated data collection, a versioned feature store, multi-model training and evaluation, a model registry, an automated CI/CD pipeline, and a live, interactive web dashboard with explainable predictions and hazard alerts based on the available current feature data.

The finished system is deployed and publicly accessible. New data and predictions continue to be collected automatically through a scheduled hourly pipeline. However, near the project deadline, the Hopsworks free-tier compute budget was exhausted, so the automated daily training and drift-check schedule was temporarily paused, while the hourly feature/prediction schedule was kept active since it does not depend on the exhausted compute budget. The latest registered model and available data continue to support the deployed dashboard, which is designed for a non-technical audience and includes plain-language health guidance, a "best time for outdoor activity" recommendation, and visual explanations of what is driving each forecast.

Live dashboard: https://aqi-predictor-jhpqumq2pnd6fud4vuzzfj.streamlit.app

Source code: https://github.com/hibaasghar30/aqi-predictor

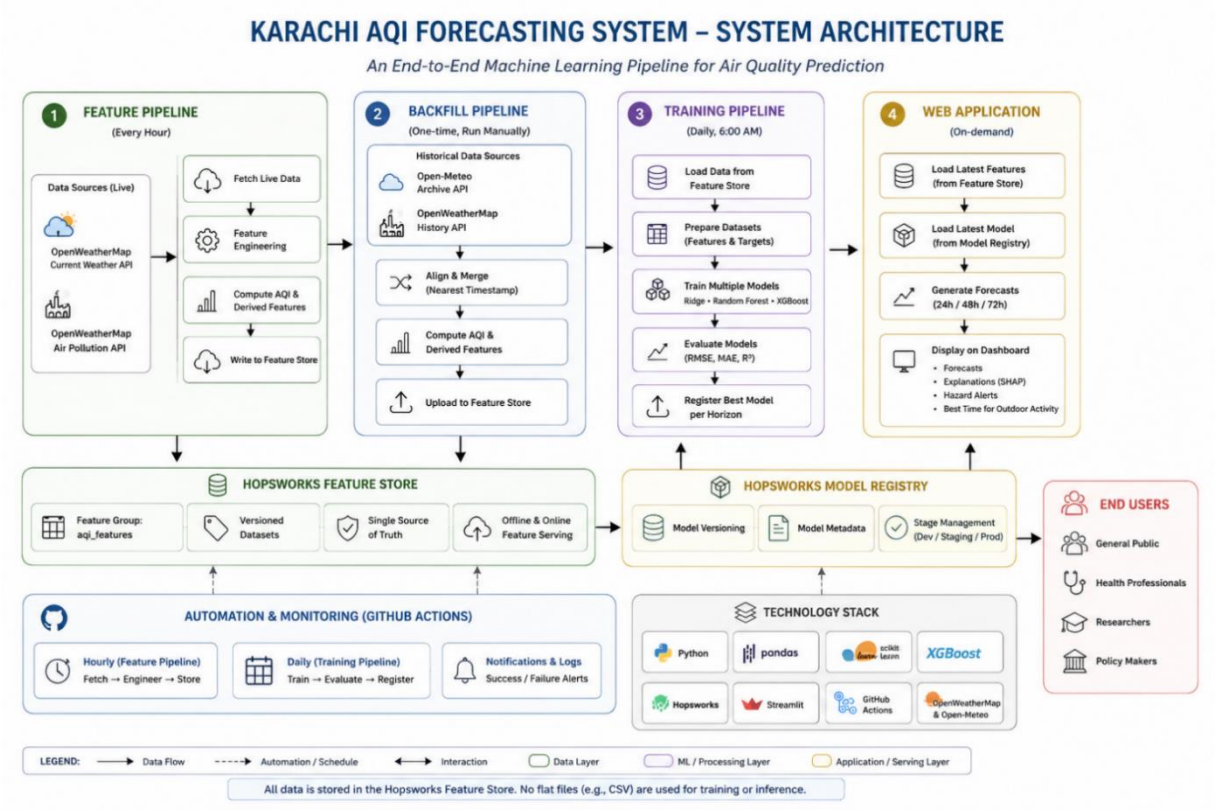
2. Project Objectives

The brief for this project asked for a serverless AQI prediction service composed of four connected pipelines, evaluated against the following goals:

- A feature pipeline that fetches raw weather and pollutant data and computes model-ready features and targets.
- A backfill process to populate historical training data.
- A training pipeline that experiments with multiple model families and evaluates them using RMSE, MAE, and R^2.
- An automated CI/CD pipeline configured to keep the feature store and models up to date through scheduled workflows.
- A web application that loads the latest model and features and presents forecasts on an interactive dashboard.
- Supporting practices: exploratory data analysis, model explainability (SHAP/LIME), and hazard alerting.

Each of these is addressed in the sections that follow, along with an honest account of the engineering decisions, trade-offs, and issues encountered while building the system.

3. System Architecture



The system is organized as four cooperating pipelines that share two central pieces of infrastructure: a Hopsworks Feature Store (for data) and a Hopsworks Model Registry (for trained models). This mirrors the serverless, MLOps-style architecture outlined in the project brief.

Pipeline	Responsibility	Trigger
Feature pipeline	Fetch live weather + pollution data, engineer features, write to the feature store	Every hour — scheduled automation (active)
Backfill pipeline	Populate historical (features, targets) for model training	One-time, run manually
Training pipeline	Train and evaluate multiple models per forecast horizon, register the best one	Daily, 6:00 AM — scheduled automation (currently paused)
Web application	Load the latest features and model, generate and display forecasts	On-demand (Streamlit Cloud)

Submission-state note: The hourly and daily schedules were used/configured during development. Near the project deadline, the daily automatic GitHub Actions cron schedule (training and drift check) was temporarily paused after the Hopsworks free-tier compute budget was exhausted. The hourly feature/prediction schedule was kept active, per the project mentor's guidance, since it continues to work without requiring the exhausted compute budget. The workflow code was preserved, and manual

workflow_dispatch triggering remains available for controlled testing of the training pipeline when Hopsworks compute becomes available.

3.1 Technology Stack

Component	Tool / Library
Weather & pollution data	OpenWeatherMap API
Historical weather backfill	Open-Meteo Archive API
Feature store & model registry	Hopsworks (free tier)
Modelling	scikit-learn (Ridge, Random Forest), XGBoost
Explainability	SHAP
Automation / CI-CD	GitHub Actions
Dashboard	Streamlit
Deployment	Streamlit Community Cloud

4. Data Collection & Feature Pipeline

Live weather and pollutant readings for Karachi are retrieved from the OpenWeatherMap API (current weather and air pollution endpoints). When the feature pipeline is enabled, each hourly run constructs a single feature row combining:

- Pollutant concentrations: PM2.5, PM10, CO, NO2, SO2, O3
- Weather conditions: temperature, humidity, pressure, wind speed
- Time-based features: hour, day, month, day of week
- Derived features: rolling AQI averages (24h / 48h / 72h) and AQI change rate

The AQI itself is derived from PM2.5 concentration using a standard AQI breakpoint conversion. Each completed row is written to a Hopsworks feature group ("aqi_features"), which acts as the single source of truth for both training and live inference — per the project brief's guidance to avoid storing data in flat files such as CSVs.

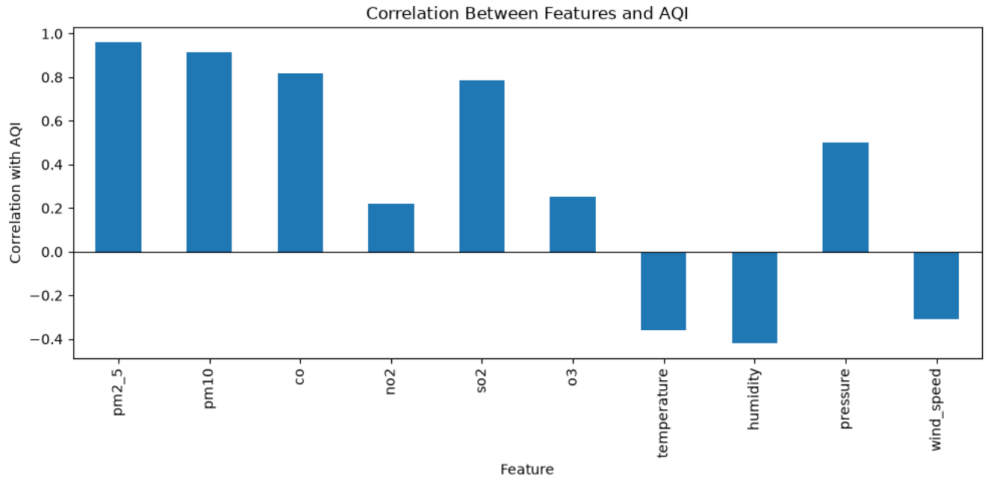
4.1 Backfill

To generate enough historical data to train the initial models, a backfill script pulls roughly two years of historical hourly weather (via Open-Meteo's archive API) and historical pollution data (via OpenWeatherMap's history endpoint, fetched in 30-day chunks to respect API limits). The two sources are aligned by nearest timestamp (within a one-hour tolerance) using pandas' merge_asof, after which AQI and AQI-change-rate are computed and the combined dataset is uploaded to the feature store in the same schema used by the live pipeline.

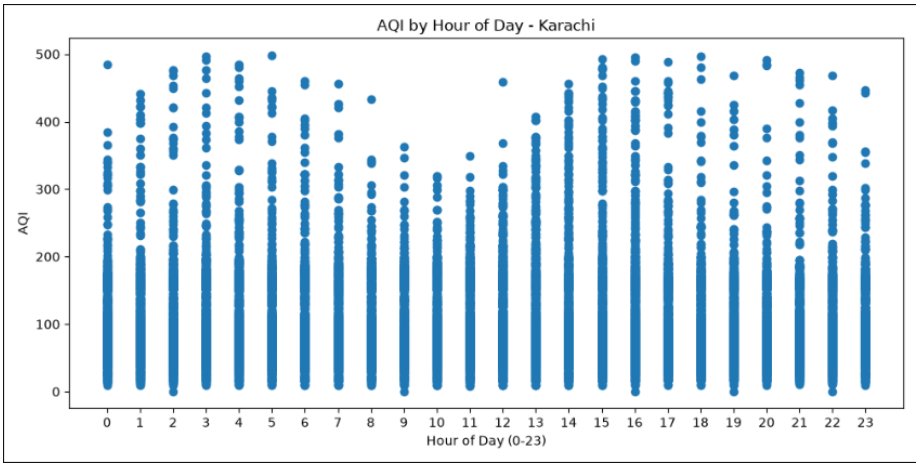
This was run once to seed the feature store with historical training data; it is not part of the ongoing automated schedule.

5. Exploratory Data Analysis

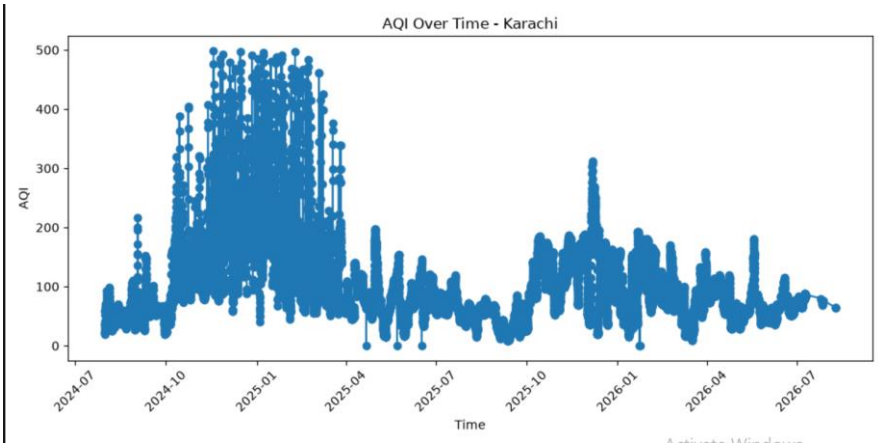
Feature Correlation with AQI



AQI by Hour of Day



AQI Over Time



Before modelling, exploratory analysis was performed on the collected data to identify trends and validate assumptions:

- AQI over time — a time-series plot of AQI across the full collection period, used to sanity-check for gaps and unexpected spikes.
- AQI by hour of day — a scatter plot examining whether AQI follows a daily cycle (e.g. rush-hour or overnight effects).
- Feature correlation with AQI — a bar chart of each raw feature's Pearson correlation with AQI, used to sanity-check which pollutants and weather variables move together with air quality before they were fed into the models.

This analysis directly informed the feature set used in training and gave early confidence that PM10 and PM2.5 in particular were meaningfully associated with overall AQI, a finding that was later confirmed independently by the SHAP explainability results (Section 8).

6. Model Training & Evaluation

Three forecasting horizons are modelled independently — 24, 48, and 72 hours ahead — since each represents a genuinely different prediction problem with its own signal-to-noise characteristics.

6.1 Models Compared

For each horizon, three candidate models were trained on an 80/20 train-test split and compared on the same held-out data:

- Ridge Regression — a regularized linear model, included as a statistical baseline.
- Random Forest Regressor — an ensemble of decision trees.
- XGBoost Regressor — a gradient-boosted tree ensemble.

The model with the lowest RMSE on the test set is automatically selected as the "best model" for that horizon and is the only one persisted to disk and registered in the Hopsworks Model Registry. In practice, different horizons selected different winning models — for example XGBoost for the 24-hour and 72-hour horizons, and Random Forest for the 48-hour horizon at various points during development — which reflects genuine, independent competition between the algorithms rather than a fixed default.

6.2 Evaluation Metrics

Each candidate is scored using all three metrics requested in the brief:

- RMSE (Root Mean Squared Error) — penalizes large errors more heavily than small ones.
- MAE (Mean Absolute Error) — the average absolute size of a prediction error, in AQI points.
- R^2 (coefficient of determination) — the proportion of variance in AQI explained by the model, from roughly 0 (no better than guessing the average) to 1 (perfect fit).

At the time of the last full evaluation, the 24-hour model achieved an R^2 of approximately 0.86, while the 72-hour model's R^2 dropped to approximately 0.35. This pattern — accuracy declining as the forecast horizon lengthens — was reviewed with the project mentor, who confirmed it is expected and acceptable, since predicting further into the future is inherently a harder problem with more accumulated uncertainty. The mentor's guidance was that this is not a defect to "fix", provided the model beats a naive persistence baseline and RMSE remains reasonably low, both of which were satisfied.

6.3 Design Decision: No Deep Learning Model

The project brief suggested experimenting with a range of models "from statistical modelling to deep learning models." This project deliberately used statistical (Ridge) and tree-based ensemble methods (Random Forest, XGBoost) rather than a deep learning model such as an LSTM or feed-forward neural network. This was a considered choice rather than an oversight: gradient-boosted trees are widely regarded as a strong, often superior choice for structured, tabular data of moderate size, whereas deep learning models typically require substantially larger datasets to outperform tree-based methods and add considerable complexity without a clear expected benefit here. This trade-off, and the reasoning behind it, is flagged explicitly as a known limitation in Section 10.

7. Model Registry & Automation (CI/CD)

Once trained, each horizon's winning model, its scaler (where required), and its metadata (best model name, feature list) are saved locally and then uploaded to the Hopsworks Model Registry, tagged with its RMSE. The live application is designed to use the latest available registered model for each horizon, with a local fallback retained for continued dashboard operation when the Hopsworks compute environment is unavailable. This allows the deployed dashboard to remain usable even when the scheduled training pipeline is paused.

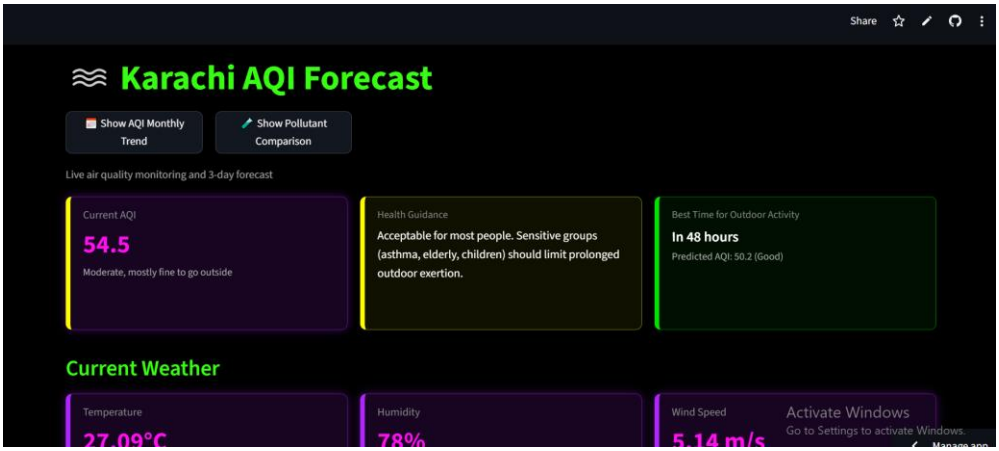
The pipeline was automated using GitHub Actions through a single workflow, `.github/workflows/pipeline.yml`. During development, the workflow was configured with two scheduled triggers: an hourly trigger for feature collection and prediction, and a daily 6:00 AM trigger for model training and drift checking. Near the project deadline, the daily cron schedule was temporarily disabled after the Hopsworks free-tier compute budget was exhausted; the hourly schedule was kept active since it does not require the exhausted compute budget. The workflow itself was preserved, and `workflow_dispatch` remains enabled so that the training pipeline can be manually triggered when Hopsworks compute becomes available.

```
on:
  schedule:
    - cron: "0 * * * *"    # every hour -> feature pipeline (active)
    # daily 6:00 AM -> training + drift check (paused at submission)
    # - cron: "0 6 * * *"
  workflow_dispatch:
```

The hourly trigger fetches live data, writes a new feature row, and generates a fresh prediction, and remains active. When enabled, the daily 6:00 AM trigger retrains the three forecast-horizon models and performs the data-drift check; this trigger is currently paused and runs only via manual workflow_dispatch. Thus, the automation remains part of the implemented system design, with only the compute-heavy daily training/drift-check schedule paused at submission because of the external Hopsworks compute limitation.

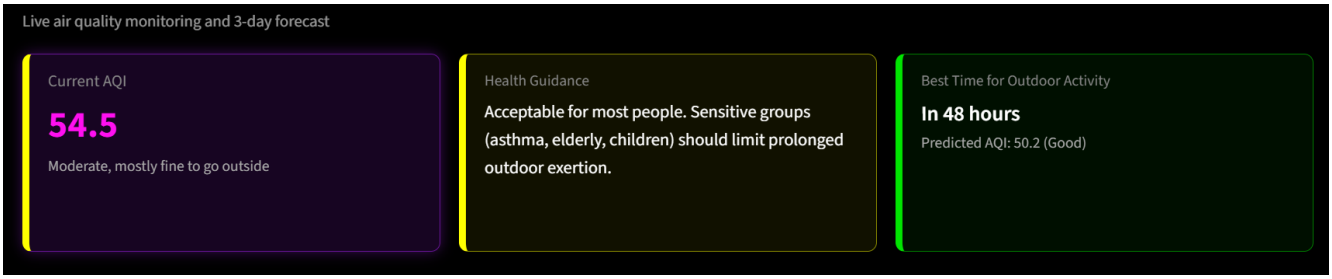
Note on a fixed issue: during development, the script invoked by GitHub Actions (predictor.py) had its executable entry point accidentally removed while refactoring for the web app, meaning the scheduled hourly job was completing "successfully" without actually collecting new data. This was identified during pre-deployment review, fixed by restoring a guarded entry point (if __name__ == "__main__":), and verified by manually re-running the script and confirming a new row was written to the feature store.

8. Web Application & Dashboard



The dashboard is built with Streamlit and deployed on Streamlit Community Cloud, per the mentor's guidance that this platform was appropriate for the project. It loads the current model and the most recent feature row on each visit and presents the following:

8.1 Live Forecast & Health Guidance

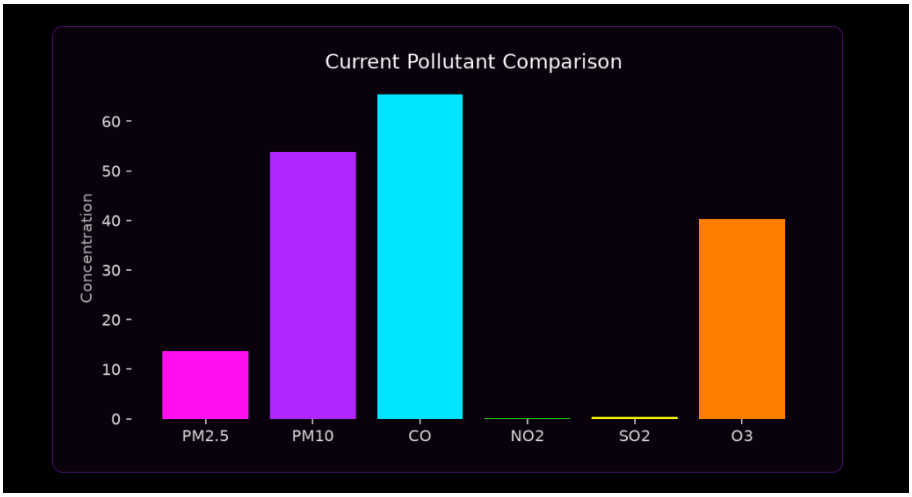


- Current AQI, with a colour-coded severity category (Good / Moderate / Unhealthy for Sensitive Groups / Unhealthy / Very Unhealthy / Hazardous).

- Plain-language health guidance translating the current AQI category into concrete advice (e.g. mask recommendations, guidance for sensitive groups).
- A "best time for outdoor activity" indicator that compares the current AQI against all three forecasts and recommends the lowest-AQI window.
- 24-, 48-, and 72-hour forecast cards, each showing the predicted AQI, its severity category, and which model produced it.

Each forecast card is paired with a SHAP (SHapley Additive exPlanations) waterfall-style chart showing the six features with the greatest influence on that specific prediction, colour-coded by whether they pushed the forecast up (red) or down (blue). This satisfies the brief's requirement for feature-importance explanations and directly supports trust in the model's output — a user (or evaluator) can see, for any given forecast, that the model is reasoning from real signals such as current AQI and PM10 levels rather than behaving as a black box.

8.2 Additional Views

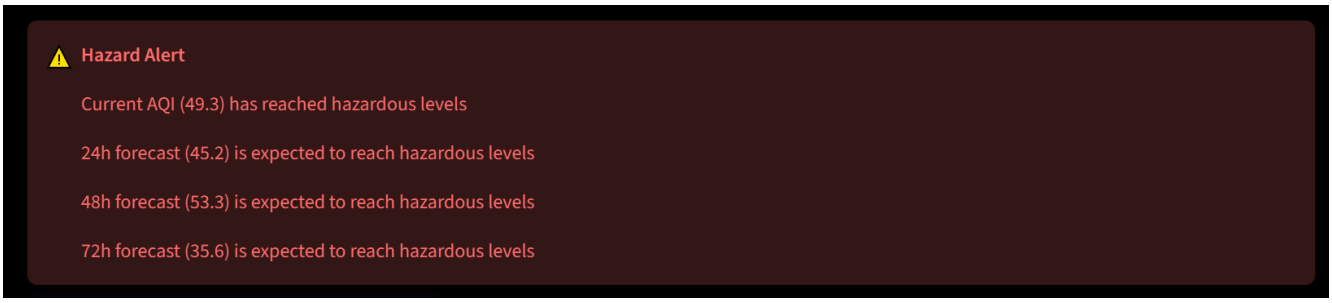


- Current weather conditions (temperature, humidity, wind speed) for context.
- Current pollutant levels (PM2.5, PM10, CO, NO2, SO2, O3) as individual cards.
- A 7-day historical trend chart with the 3-day forecast plotted alongside it.
- A monthly AQI trend chart with a year selector, letting a user compare seasonal patterns across the full collection period.
- A live pollutant comparison chart, showing all six pollutants' current concentrations side by side.

The monthly trend and pollutant comparison views are implemented as separate in-app "pages" using Streamlit's session state, so the main dashboard stays uncluttered and a user can navigate back and forth with a single button.

8.3 Hazard Alerts

If the current reading or any forecast exceeds a defined hazard threshold (AQI ≥ 150), the dashboard displays a prominent warning banner and the alert is logged to the feature store, satisfying the brief's requirement for hazardous-AQI alerting.



9. Deployment

The application is deployed on Streamlit Community Cloud, connected directly to the project's public GitHub repository. Deployment is continuous: any commit pushed to the main branch is automatically detected and redeployed within a couple of minutes, so the live app always reflects the latest code.

9.1 Deployment Issues Resolved

Getting from a working local environment to a working cloud deployment surfaced several dependency-resolution issues that are common in real-world MLOps work and are documented here for completeness:

Issue	Root Cause	Resolution
Package not found	hopsworks==5.0.3 requires Python ≥3.10, <3.14; Cloud environment defaulted to 3.14	Explicitly set the deployment's Python version to 3.12

Issue	Root Cause	Resolution
Conflicting dependency	A hard-pinned protobuf==7.35.1 in requirements.txt directly conflicted with hopsworks' own required range (<5.0.0)	Removed the redundant pin and let pip resolve protobuf automatically
Unsatisfiable dependencies	streamlit==1.61.1 requires protobuf ≥5.26; hopsworks==5.0.3 requires protobuf <5.0 — no overlapping version exists	Downgraded to streamlit==1.58.0, whose protobuf range (≥3.20, <8) is compatible with both hopsworks and the rest of the stack
App failed at first load (KeyError)	Environment secrets (API keys) had not yet been configured in Streamlit Cloud	Added OPENWEATHER_API_KEY, HOPSWORKS_API_KEY, and HOPSWORKS_PROJECT_NAME via the platform's encrypted Secrets manager

Each of these was diagnosed directly from the platform's build logs (which report the exact conflicting version ranges) rather than by trial and error, and each fix was verified locally before being pushed, to keep the local and deployed environments in sync.

9.2 Operational Issue: Hopsworks Free-Tier Budget

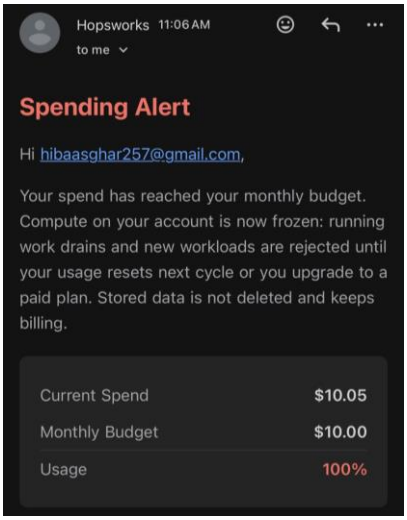


Figure. Hopsworks free-tier spending alert showing that the $10.00 monthly compute budget had been exhausted.

Partway through the project, the automated GitHub Actions pipeline began failing after the Hopsworks free-tier project reached its monthly compute budget. The Hopsworks account notification reported "Current Spend: $10.05, Monthly Budget: $10.00, Usage: 100%", indicating that further compute-heavy operations were unavailable. Hopsworks confirmed that stored data is not deleted and continues to be billed against the same cap, while the compute freeze can prevent operations such as materialization and training.

This was raised directly with the project mentor, who confirmed the diagnosis and gave the following guidance:

"Yes, that is almost certainly the reason. Once Hopsworks compute is frozen, materialization and training jobs will fail even if your dashboard still loads. Pause the daily pipeline for now. Keep your

hourly GitHub job running if it still works without compute. Use your latest registered model and local fallback for predictions until the budget resets or you move to a new personal Hopsworks account. Do not rebuild the full setup before the deadline. Document this in your report with the spending alert screenshot."

Following this guidance, the GitHub Actions workflow (`.github/workflows/pipeline.yml`) was updated to temporarily disable the daily automatic cron schedule. The daily training/drift-check schedule (`0 6 * * *`) was commented out rather than deleted, preserving the original automation for future restoration, while the hourly feature/prediction schedule (`0 * * * *`) was kept active since it does not depend on the exhausted compute budget.

- The daily cron trigger (0 6 * * *) was commented out — not deleted — so it can be restored in a single line once the budget resets or a paid plan is adopted.
- The training and drift-check steps now only run on a manual workflow_dispatch trigger, rather than automatically every day.
- The automatic hourly feature-collection and prediction schedule was kept active, per the mentor's guidance, since it continues to work without requiring the exhausted Hopsworks compute budget.

The deployed Streamlit dashboard remained functional because it uses the available registered model and existing feature data/local fallback without requiring the scheduled training pipeline to run. This allowed the application to remain publicly accessible while avoiding further failed scheduled jobs against the exhausted Hopsworks compute budget.

9.3 Mobile Readability

Streamlit automatically stacks side-by-side columns vertically on narrow screens. After testing the deployed app on a phone, card spacing (margin-bottom) was increased across all custom-styled cards so that stacked content on mobile remains legible and well separated, without changing the desktop layout.

10. Known Limitations

In the interest of an accurate and complete report, the following limitations are noted explicitly rather than glossed over:

- No deep learning model was trained; see the justification in Section 6.3.
- Forecast accuracy decreases with horizon length (R^2 of ~0.86 at 24h vs. ~0.35 at 72h). This was reviewed and accepted by the project mentor as an expected characteristic of longer-horizon forecasting, not a defect.
- RMSE, MAE, and R^2 are currently printed to the console during training but are not yet persisted to the model metadata file or surfaced on the dashboard. Extending the training script to save these alongside the existing best_model metadata (and adding a small "model performance" view to the dashboard) is a natural next step, but was deprioritised in favour of completing deployment within the project timeline.
- Automated data collection has occasional gaps corresponding to periods when the pipeline was not yet fully automated during development; the 7-day and monthly trend charts reflect only the data that was actually collected.

- The automated daily training/drift-check schedule is currently paused after the Hopsworks free-tier account reached its monthly compute budget; the hourly feature/prediction schedule remains active since it does not require the exhausted compute budget. The training/drift-check workflow code has been preserved and can be restored when Hopsworks compute becomes available. The dashboard remains functional using the available registered model, existing feature data, and local fallback.

11. Mentor Clarifications

The following clarifications were sought from the project mentor during development and directly shaped the final design:

Question	Guidance Received
Should AQI be predicted as an hourly forecast or a daily average, for each of the next 3 days?	Daily average per horizon — the approach used throughout this project.
Is Streamlit Community Cloud an acceptable deployment target?	Yes.
Is a large drop in R^2 between the 24h and 48h/72h models a problem?	No — a decreasing R^2 across longer horizons is expected. The bar is that predictions remain directionally accurate, beat a naive persistence baseline, and keep RMSE low.
Should raw data be stored in local CSV files?	No — data must be stored in a proper deployed feature store (Hopsworks, in this project), not local CSVs, to keep the pipeline reproducible and production-like.
The Hopsworks free-tier compute budget was reached and the daily pipeline began failing — is this the cause, and how should it be handled before the deadline?	Confirmed as the cause. Pause the daily pipeline, keep the hourly job running if it still works, rely on the latest registered model and dashboard until the budget resets or a new account/plan is used, avoid rebuilding the full setup before the deadline, and document the issue in the report with the spending alert screenshot.

Final implementation decision: The final repository state matches the mentor's guidance directly — the daily training/drift-check schedule was paused, while the hourly feature/prediction schedule was kept active since it continues to work without requiring the exhausted compute budget. This avoided repeated scheduled failures on the training/drift-check job while preserving fresh hourly data collection and the workflow code for future restoration. The latest registered model and local fallback continued to support the deployed dashboard.

12. Conclusion & Future Work

This project delivers a working and deployed AQI forecasting system for Karachi that addresses the major requirements of the original brief: a feature pipeline, a historical backfill process, a multi-model training pipeline evaluated with RMSE/MAE/R^2, a model registry, exploratory data analysis, SHAP-based explainability, hazard alerting, and a live interactive dashboard connected to a shared feature store and model registry. The system was configured with an automated CI/CD workflow for hourly feature collection and prediction and daily training and drift checking. Near submission, however, the daily training/drift-check trigger was temporarily paused after the Hopsworks free-tier compute budget was exhausted, while the hourly feature/prediction trigger was kept active. The workflow code, latest registered models, and dashboard were preserved, allowing the deployed application to remain functional while clearly documenting the operational limitation.

Given more time, the most valuable next steps would be:

- Persisting RMSE/MAE/R^2 to model metadata and surfacing a "model performance" view on the dashboard.
- Experimenting with a deep learning model (e.g. an LSTM) as an additional candidate in the training comparison, purely to validate whether it meaningfully outperforms the current tree-based approach on this dataset.
- Backfilling the small gaps in the historical data caused by the pipeline not yet being fully automated during early development.